

CRUD: CREATE (POST), READ (GET), UPDATE (PUT/PATCH), DELETE (DELETE)

Uma API, Interface de Programação de Aplicações (do inglês Application Programming Interface) é um conjunto de serviços que foram implementadas em um software que são disponibilizados para que outros aplicativos possam utilizá-los diretamente de forma simplificada; sem envolver-se em detalhes da implementação do software.

REST (Representational State Transfer) é um estilo de arquitetura para sistemas distribuídos baseado na comunicação entre cliente e servidor por meio do protocolo HTTP. Suas principais características incluem comunicação stateless, interface uniforme, identificação de recursos por URIs e suporte a cache.

Possui um modelo cliente-servidor, em que cliente e servidor são entidades independentes que se comunicam por meio de uma interface uniforme baseada no protocolo HTTP.

A comunicação é stateless, ou seja, o servidor não mantém o estado da sessão do cliente entre as requisições. Cada requisição deve conter todas as informações necessárias para seu processamento. O estado pode existir tanto no cliente (cookies, armazenamento local etc.) quanto no servidor (armazenamento em memória, banco de dados etc.), mas nenhuma requisição depende das anteriores.

A interface uniforme exige que os recursos sejam identificados por URIs, manipulados por meio de suas representações (como JSON), que cada requisição contenha todas as informações necessárias para seu processamento e que a aplicação pode fornecer links indicando as próximas ações disponíveis ao cliente, sem que ele precise conhecer previamente a API.

Os recursos podem ser armazenados em cache pelo cliente para evitar transferências de dados desnecessárias e melhorar o desempenho.

O verbo GET é usado para obter dados. As requisições GET devem ser idempotentes, ou seja, realizar várias requisições iguais produz o mesmo efeito no servidor, desde que o recurso não seja alterado por outra operação. Em caso de sucesso, retorna uma representação em JSON e um código de resposta HTTP de 200 (OK). Em caso de erro, pode-se retornar um código de resposta 404 (NOT FOUND) ou 400 (BAD REQUEST).

Ex.: Procurar um vídeo no youtube

O verbo POST é mais frequentemente utilizado para criar um registro, mas também para tudo aquilo que não é coberto pelos outros métodos específicos. Não é idempotente. Na criação bem-sucedida, retorna o JSON do registro criado e código de resposta HTTP 201 (Created).

Ex.: Criar uma playlist de música no spotify

O verbo PUT é utilizado para atualizar ou criar um registro. Todos os dados de um registro são agrupados e são enviados para a mesma URL desse registro, substituindo aquele registro. Se não havia dados ali, é criado um registro, se existia, é atualizado. Todas as requisições de PUT devem ser idempotentes. Na atualização bem-sucedida, retorna o JSON do registro alterado e código de resposta HTTP 200 (OK).

Ex.: Atualização de cadastro

O verbo DELETE é usado para deletar um registro. Todas as requisições de DELETE devem ser idempotentes. Na exclusão bem-sucedida, devolve o status HTTP 204 (No Content) quando não há corpo ou 200 (OK) se houver.

Ex.: Deletar um post no instagram

O verbo PATCH é usado para atualizar parcialmente um registro, sem a necessidade de enviar todos os atributos. Enviar somente o que de fato precisa ser alterado. Ele não precisa ser idempotente, mas geralmente é implementado dessa forma. Na atualização bem-sucedida, retorna o

JSON do registro alterado e código de resposta HTTP 200 (OK) ou 204 (No Content) quando a operação é realizada com sucesso, mas o servidor não retorna um corpo de resposta.

Ex.: Atualização do email de um cadastro